

Project Report

1. Introduction

Every year, over a million students in India sit for JEE Main — one of the most competitive engineering entrance exams in the world. After the exam, the most common question every student ask is: "What rank will I get?" Right now, there is no reliable free tool that gives a student a concrete, ML-backed answer to that question.

This project, JEE Main Rank Predictor, is a Python application that takes a student's subjectwise marks (Maths, Physics, Chemistry), their category (General or Reserved), and their gender — and predicts their approximate JEE Main rank along with their percentile. The prediction is backed by a trained Gradient Boosting model, not a simple lookup table.

2. Problem Statement

The problem is straightforward but genuinely useful: after finishing the exam, a JEE aspirant has no reliable way to know where they stand without waiting weeks for official results. The existing rank estimators available online either:

- Use simplistic linear formulas that don't account for the actual score distribution
- Are not broken down by category or gender, which affects the real rank significantly
- Don't show you how your marks compare to the larger pool of students
- Give you a single number with no context — no range, no percentile, no charts

This project directly addresses all four of these gaps by building a machine learning model trained on a statistically realistic synthetic dataset that mirrors the actual JEE score distribution.

3. Why This Problem Matters

JEE Main results determine admission to NITs, IITs, and other centrally funded technical institutions. The category-based reservation system means a student's rank in their own category can be very different from their general rank. Getting this estimate wrong — even by a few thousand ranks — can lead to wrong college choices or unnecessary panic.

A tool that gives a realistic, ML-powered estimate helps students make better decisions: whether to apply for JEE Advanced, which colleges to target, and whether to consider a drop year. It is a small tool with real impact for millions of students every year.

4. Approach and Solution

4.1 Dataset Generation

Since real JEE mark-to-rank data is not publicly available in a clean, downloadable format, the project generates its own realistic synthetic dataset using statistical simulation. The dataset is produced entirely in-memory at runtime — no external files needed.

The simulation generates 12,000 records using a normal distribution centred around a mean total score of 110 (out of 300), with a standard deviation of 55. This closely mirrors the actual

JEE bell curve where most students score in the 60–160 range and very few score above 250. Subject-wise marks (Maths, Physics, Chemistry) are derived from this total with controlled variance so they stay realistic and consistent.

4.2 Feature Engineering

Beyond raw marks, the model uses nine engineered features for each student record:

- Individual subject marks: Maths, Physics, Chemistry
- Total marks
- Total marks squared (captures non-linear rank compression at top scores)
- Log of total marks (handles the skewed rank distribution)
- Maths minus Physics difference
- Maths minus Chemistry difference
- Sum of all pairwise subject differences (measures inconsistency)

4.3 Model Training

Four separate Gradient Boosting Regressor models are trained — one each for General Male, General Female, Reserved Male, and Reserved Female. This split is intentional: rank lists in JEE are separate for each category and gender (for female supernumerary seats), so a combined model would produce less accurate results.

Each model uses 200 estimators, a learning rate of 0.08, max depth of 5, and 85% subsampling. The dataset is split 85/15 for training and testing. Typical R^2 scores are above

0.97, meaning the model explains over 97% of the variance in rank from marks alone.

4.4 Prediction and Output

Once trained, the model accepts a student's marks and demographic inputs and returns their predicted category rank, an estimated general rank (for reserved category students, a factor of 1.8 is applied to approximate the general rank equivalent), a $\pm 15\%$ confidence range, and an approximate percentile.

All predictions are saved to a local CSV history file so students can track multiple attempts or mock test results over time.

4.5 Visualisation

The application generates two types of charts using Matplotlib with a dark theme:

- **Dataset Overview:** Shows the marks distribution bell curve for General vs Reserved and Male vs Female categories, subject-wise average marks by year, and the synthetic rank distribution.
- **Prediction Analysis:** Shows a scatter plot of all students with the user's position highlighted with a red star, a subject-wise bar chart comparing the user's marks against the dataset average, and average marks by year broken down by category.

5. Key Decisions Made

Decision	Reason
Synthetic dataset instead of scraping	No clean public dataset exists. Synthetic data lets the project run standalone without any data file dependency.
Four separate models by category and gender	JEE ranks are computed separately per category. One combined model would ignore demographic-specific score distributions.
Gradient Boosting over linear regression	The rank vs marks relationship is non-linear (top ranks compress massively at high scores). GBR handles this naturally.

History saved to CSV	Students typically run multiple predictions after mock tests. A history file lets them track progress without a database.
----------------------	---

6. Challenges Faced

6.1 No Real Dataset

The biggest challenge was the absence of a usable real-world dataset. JEE's NTA does not publish mark-wise rank data in a clean format. I considered scraping rank predictor websites but those are unreliable and legally questionable. The solution was to simulate the data using the known statistical properties of JEE — normal distribution, subject-wise correlation, category-wise splits — so the model's behaviour would be realistic even without real data.

6.2 Non-Linear Rank Compression

JEE ranks are highly non-linear. The difference between rank 1 and rank 1000 might be just 15 marks, while the same 15-mark gap at the lower end of the distribution might span 50,000 ranks. Simple linear regression completely failed here. Adding polynomial features (total^2) and $\log(\text{total})$ as inputs fixed most of this and the GBR model handled the rest.

6.3 Matplotlib Backend on Different Systems

Running charts on different operating systems (Windows, Mac, Linux) required handling the Tkinter and Qt5 backends gracefully. The final solution detects the available backend at runtime and falls back to saving the chart as a PNG file if no display backend is available — which also makes it compatible with headless environments like servers.

6.4 Reserved Category Rank Conversion

The relationship between a reserved category rank and the equivalent general rank position is not published officially. I studied historical cut-off data and settled on a factor of 1.8 as a reasonable approximation — meaning a Reserved rank of 1000

roughly corresponds to around rank 1800 in the general pool. This is acknowledged as an approximation in the output.

7. What I Learned

- How to simulate realistic datasets when real data is unavailable — a skill that is genuinely important in applied ML.
- The importance of feature engineering. The model's accuracy jumped significantly when I added squared and log features compared to just using raw marks.
- How tree-based ensemble models like Gradient Boosting handle non-linear relationships far better than linear methods for rank-prediction tasks.
- Practical experience with Matplotlib's backend system and how to write code that works regardless of the user's display environment.
- How to structure a Python project with a clean menu-driven interface, persistent local history, and modular functions.
- The value of breaking a problem into small testable pieces — training the models, then predicting, then visualising, rather than trying to build everything at once.

8. Limitations and Future Scope

8.1 Current Limitations

- The dataset is synthetic, so while it mirrors statistical properties of JEE, it is not trained on actual student data.
- The reserved-to-general rank conversion uses a fixed factor rather than a yearspecific or subject-specific one.
- State-wise rank (used for some state-level counselling) is not predicted.

- No web interface — it runs only from the command line.

8.2 What Could Be Added Next

- A web interface using Flask or Streamlit so students can access it from a browser.
- Integration with actual historical NTA data if it ever becomes publicly available.
- College predictor based on predicted rank using JoSAA cutoff data.
- Support for JEE Advanced rank prediction as a separate module.

9. Conclusion

JEE Main Rank Predictor is a small, purposeful tool built to answer a real question that millions of students in India ask every year. It applies machine learning concepts covered in this course — feature engineering, model training, evaluation metrics, and data visualisation — to a problem that has genuine practical value.

The project is self-contained, runs without any external data files, and gives students a useful rank estimate along with visual context about where they stand in the broader distribution. It is not perfect, but it is meaningfully better than the back-of-envelope calculations most students currently use.

Building this taught me that the hardest part of a real ML project is usually not the algorithm — it is getting the data right, handling edge cases gracefully, and making the output actually useful to the person who will use it.